ÉTUDE DE CAS — STRIPE

LIVRABLE 7

Stratégie d’intégration de l’apprentissage automatique

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l’infrastructure de données et de calcul pour l’IA

Ce livrable décrit comment les modèles de Machine Learning sont intégrés à l'architecture de données de Stripe, et en particulier aux bases NoSQL. Il couvre les trois cas d'usage demandés (détection de fraude en temps réel, personnalisation client et analyse prédictive), ainsi que les étapes nécessaires à leur fonctionnement : extraction des features, déploiement des modèles et suivi de leurs performances.

1. Cas d'usage

Cas d'usage	Type de modèle	Mode	Données utilisées	Métrique suivie
Détection de fraude	Classification (gradient boosting, ex. XGBoost)	Temps réel (< 100 ms)	Transaction en cours + features Redis	Precision et Recall
Personnalisation client	Segmentation (K-means sur récence, fréquence et montant des achats)	Batch hebdomadaire, utilisé en temps réel	Historique des achats et de l'activité	Cohérence des segments
Analyse prédictive	Prévision de séries temporelles	Batch mensuel	Table agg_revenue_daily (Redshift)	Erreur de prévision (MAPE)

Pour la fraude, on ne suit pas la justesse globale (accuracy) : les fraudes représentent une très petite partie des transactions. Un modèle qui ne détecterait jamais de fraude aurait une justesse de plus de 99 % tout en étant inutile. On suit donc la précision (part des alertes qui sont de vraies fraudes) et le recall (part des fraudes détectées).

2. Rôle des bases NoSQL dans le Machine Learning

Les deux bases NoSQL de l'architecture ont chacune un rôle dans le Machine Learning :

- **Redis (Amazon ElastiCache)** sert de stockage des features en temps réel. Pour chaque client et chaque marchand, il contient les features déjà calculées (nombre de paiements dans la dernière heure, montant moyen sur 30 jours, pays habituels, segment client…). Le service d'inférence les lit en quelques millisecondes au moment du paiement.
- **DocumentDB** conserve les données comportementales (sessions, clics, logs), à partir desquelles une partie des features est calculée. Il enregistre aussi chaque prédiction du modèle, transmise par Kafka, dans une collection fraud_predictions : transaction, score, décision, version du modèle utilisée. Ces données servent ensuite à suivre les performances du modèle.

3. Extraction des features

Les features sont calculées à deux endroits, avec exactement les mêmes formules :

- **En continu**, par le service de calcul des features (ECS, Elastic Container Service), à partir des événements Kafka. Le résultat est écrit dans Redis et sert à l'inférence.
- **En batch**, par Airflow, à partir de l'historique du Data Lake S3. Le résultat sert à l'entraînement.

Feature	Calcul	Utilité
Nombre de paiements sur la dernière heure	Compteur par client	Repérer une série de paiements rapprochés
Écart au montant habituel	Montant / montant moyen sur 30 jours	Repérer un montant inhabituel
Nouveau pays	Pays de l'IP différent des pays habituels	Repérer une utilisation depuis un autre pays
Nouveau moyen de paiement	Première utilisation de cette carte	Repérer une carte volée
Cartes différentes dans la session	Compteur par session (DocumentDB)	Repérer des tests de cartes
Taux de fraude du marchand	Part de fraudes sur 30 jours	Tenir compte du risque du marchand

Si les formules étaient différentes entre l'entraînement et l'inférence, le modèle recevrait en production des données différentes de celles sur lesquelles il a appris, et ses performances baisseraient sans raison apparente.

4. Détection de fraude en temps réel

1. La transaction arrive dans le service d'inférence (API FastAPI déployée sur Amazon ECS).
2. Le service récupère dans Redis les features du client et du marchand, puis les combine avec les informations de la transaction (montant, pays, appareil…).
3. Le modèle calcule un score de fraude entre 0 et 1.
4. Selon le score, le paiement est accepté ou refusé. La transaction est enregistrée dans Aurora PostgreSQL avec son score et son statut, et une alerte est envoyée à l'équipe fraude si le paiement est suspect.
5. La prédiction est publiée dans Kafka, puis enregistrée dans DocumentDB pour le suivi du modèle. Cet envoi se fait en parallèle et ne rallonge pas le temps de réponse.

L'ensemble doit se faire en moins de 100 ms. Si le service d'inférence ne répond pas dans ce délai, des règles simples de secours prennent le relais (montant maximum, pays bloqués), et la transaction est marquée « non scorée » (score de fraude vide dans l'OLTP) pour être réanalysée ensuite.

5. Personnalisation client et analyse prédictive

- **Personnalisation :** chaque semaine, un modèle de segmentation regroupe les clients selon leur récence, leur fréquence et le montant de leurs achats (par exemple : nouveaux clients, clients réguliers, gros acheteurs, clients inactifs). Le segment de chaque client est écrit dans Redis. Il peut ainsi être utilisé en temps réel pour adapter l'expérience de paiement, par exemple en proposant en premier le moyen de paiement préféré du client.
- **Analyse prédictive :** chaque mois, un modèle de prévision est entraîné sur l'historique des revenus (table 'agg_revenue_daily' de Redshift) pour estimer les revenus des mois suivants, par marchand et au global. Les prévisions sont enregistrées dans Redshift et affichées dans les tableaux de bord BI.

6. Entraînement et déploiement des modèles

Le cycle de vie des modèles est orchestré par Airflow :

1. **Préparation des données :** extraction de l'historique depuis le Data Lake S3, calcul des features et association avec la réalité observée. Pour la fraude, une transaction est considérée comme frauduleuse lorsqu'elle a fait l'objet d'une rétrofacturation pour motif de fraude (le titulaire de la carte a contesté le paiement auprès de sa banque), ou lorsqu'elle a été confirmée comme frauduleuse par l'équipe fraude.
2. **Entraînement du modèle sur une instance EC2** créée pour l'occasion, puis supprimée à la fin de l'entraînement (dimensionnement présenté au livrable 1, section 6).
3. **Enregistrement dans MLflow :** chaque modèle est enregistré avec ses paramètres, ses performances et sa version. Le prétraitement des données est enregistré avec le modèle, ce qui garantit qu'il sera appliqué de la même façon en production.
4. **Validation :** le nouveau modèle est comparé au modèle en production sur les mêmes données de test. Il n'est retenu que s'il est meilleur.
5. **Déploiement :** le nouveau modèle est chargé dans le service d'inférence et remplace l'ancien. En cas de problème, on peut revenir rapidement à la version précédente, conservée dans MLflow.

Le modèle de fraude est réentraîné chaque semaine. Un réentraînement plus fréquent apporterait peu, car les fraudes ne sont confirmées qu'avec plusieurs semaines de décalage, et le contexte récent est déjà fourni en temps réel par les features de Redis.

7. Suivi et mise à jour des modèles

- **Performances :** à partir des prédictions enregistrées dans DocumentDB et des fraudes confirmées ensuite, la Precision et le Recall du modèle sont recalculés régulièrement et suivis dans MLflow.
- **Dérive des données :** un traitement Airflow compare chaque jour la distribution des features et des scores avec celle des données d'entraînement. Un changement important signifie que le comportement des clients ou des fraudeurs a évolué.
- **Fonctionnement du service :** Amazon CloudWatch suit la latence et les erreurs du service d'inférence et déclenche une alerte si les 100 ms sont dépassées.

- **Mise à jour :** si les performances baissent ou si une dérive est détectée, Airflow lance un réentraînement sans attendre le réentraînement hebdomadaire.
- **Conformité :** les features ne contiennent aucun numéro de carte ni donnée d'identité (nom, email, téléphone). Les clients sont identifiés uniquement par leur identifiant, conformément au livrable 6.